

Graph DB

What is a Graph Database?

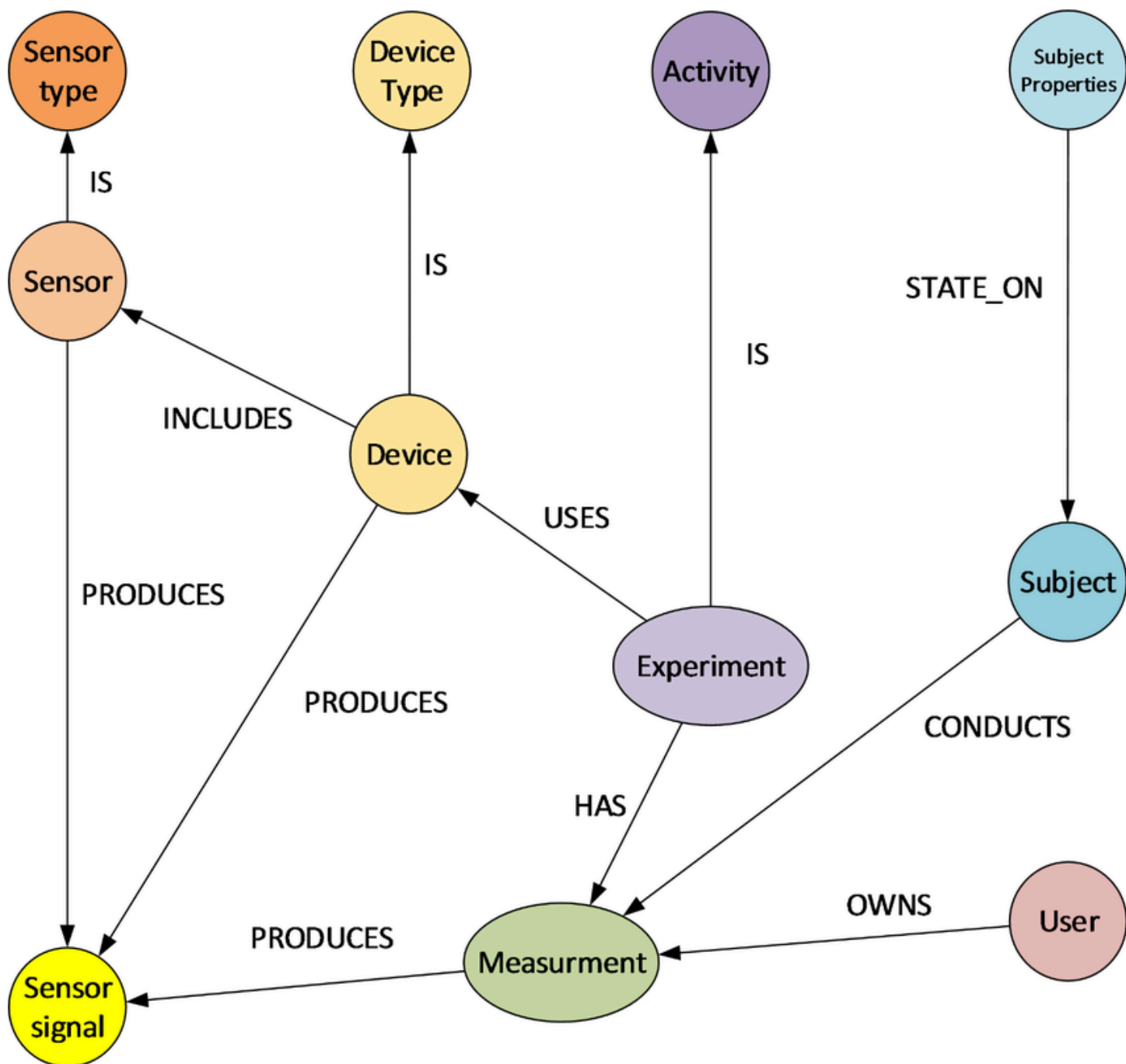
A **Graph Database** is a type of NoSQL database that stores data as **nodes (entities)** and **relationships (edges)** instead of tables and rows.

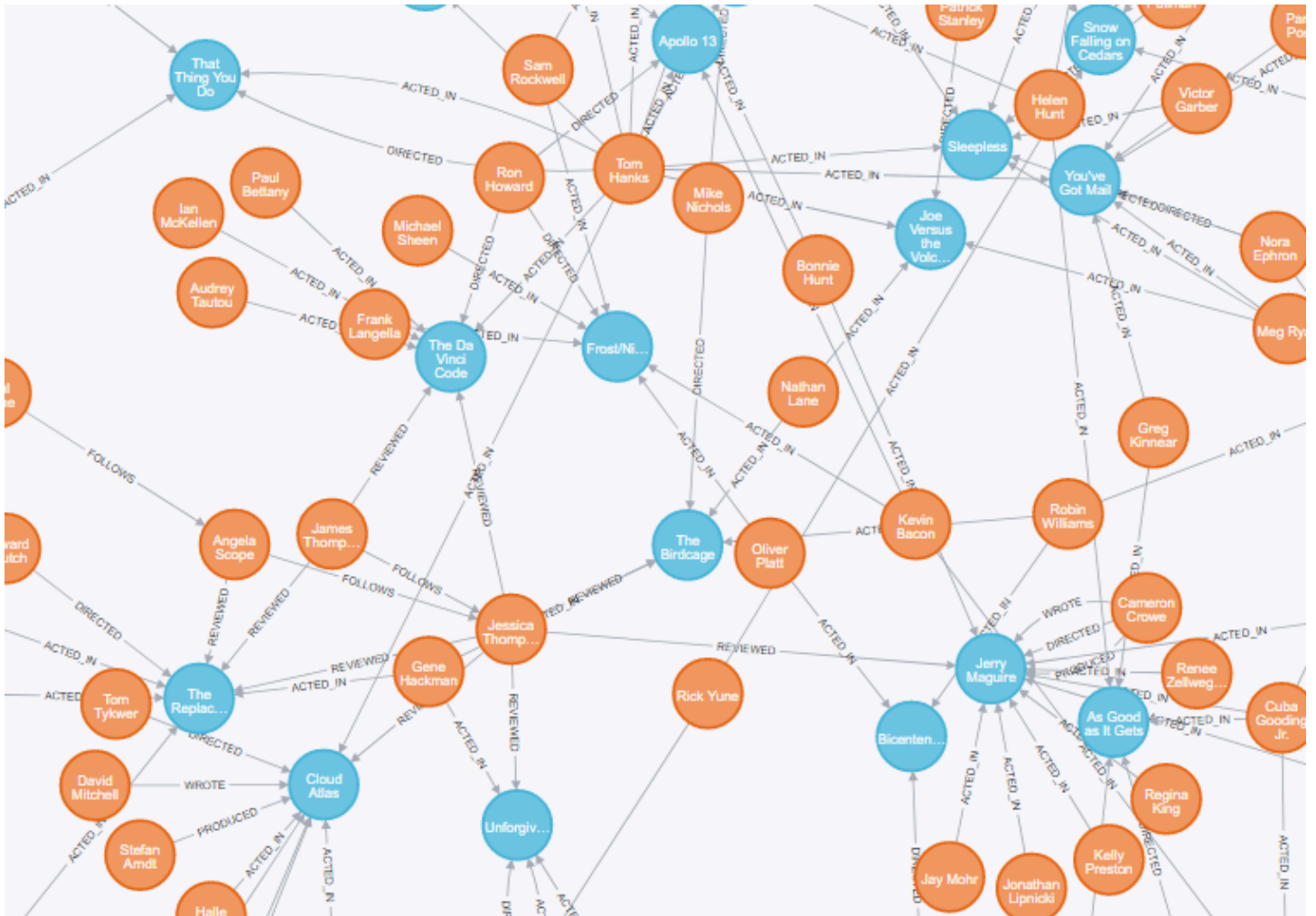
Instead of thinking in rows and columns like in MySQL or PostgreSQL, you think in terms of:

- **Nodes** → Real-world objects (User, Product, City)
- **Relationships** → Connections between nodes (FRIEND_OF, BOUGHT, LIVES_IN)
- **Properties** → Key-value data stored on nodes and relationships

Popular graph database: Neo4j

Basic Structure





1.Nodes

Represent entities.

Example:

(User {name: "Vijay", age: 22})

(Product {name: "Laptop"})

2.Relationships

Connect nodes and have direction.

(Vijay)-[:BOUGHT]->(Laptop)

Relationships can also have properties:

[[:BOUGHT] {date: "2026-02-01"}]

How Graph DB Works Internally (Deep Level)

Now let's go inside the engine.

1.Storage Model (Physical Layer)

Unlike relational DBs that store data in tables, graph DBs store:

- Node Records
- Relationship Records
- Property Records

Node Record Contains:

- Node ID
- Pointer to first relationship
- Pointer to properties

Relationship Record Contains:

- Relationship ID
- Start Node ID
- End Node ID
- Type (FRIEND, BOUGHT)
- Pointer to next relationship

This is called **Index-Free Adjacency**

What is Index-Free Adjacency?

Each node directly points to its connected relationships.

So traversal becomes:

Node → Direct pointer → Relationship → Next Node

No expensive JOIN.

2.Why It Is Faster Than Relational JOINS

In relational DB:

If you want:

Find friends of friends of Vijay

It becomes multiple JOIN operations.

JOIN complexity grows with data size.

In Graph DB:

Start at Vijay node

→ Follow FRIEND edge

→ Follow FRIEND edge again

Traversal cost depends on connected nodes, not total DB size.

That's why graph DB scales well for connected data.

3.Query Execution Engine

Example in Neo4j (Cypher query):

```
MATCH (u:User {name:"Vijay"})-[:FRIEND]->(f)
RETURN f
```

Internally:

1. Use index to find Vijay node
2. Follow pointer to relationship list
3. Filter relationship type = FRIEND
4. Return connected nodes

No table scan.

4.Indexing in Graph DB

Graph DB still uses:

- B-Tree indexes
- Hash indexes

Used only for:

- Finding starting node quickly
- Searching properties

After that → traversal handles everything.

5.Traversal Algorithm Internally

Graph DB engine uses:

- DFS (Depth First Search)
- BFS (Breadth First Search)
- Dijkstra (Shortest path)
- A* Algorithm

For example:

Find shortest path between A and B

Internally:

- Use priority queue
- Maintain visited set
- Calculate weights

Graph DB has built-in graph algorithms.

6.Memory Management

Graph DB performance heavily depends on:

- Page Cache
- Relationship pointers loaded in RAM
- Hot subgraphs in memory

Because traversal requires pointer chasing, RAM speed matters a lot.

7.ACID in Graph DB

Modern graph DBs like Neo4j provide:

- Atomicity
- Consistency
- Isolation
- Durability

Internally they use:

- Write-ahead logging
- Locking
- MVCC (in some engines)

Internal Comparison: Graph vs Relational

Feature	Relational DB	Graph DB
Storage	Tables	Nodes + Edges
Relationship	Foreign Key	Direct Pointer
Join Cost	High	No Join
Best For	Structured tabular data	Highly connected data
Example	MySQL	Neo4j

When Graph DB Is Best

Graph DB is powerful when relationships are primary.

Real Use Cases:

- Social networks
- Recommendation systems
- Fraud detection
- Knowledge graphs
- Network routing

Example companies:

- Facebook (social graph)
- LinkedIn (connection graph)
- Google (knowledge graph)

Types of Graph Models

1. Property Graph (Neo4j style)

Nodes + relationships + properties.

2. RDF Graph (Semantic Web)

Used in triple format:

Subject → Predicate → Object

Used in knowledge bases.

Internal Architecture Flow

Client Query



Query Parser



Execution Planner



Traversal Engine



Storage Engine (Node + Relationship records)

Final Understanding

Relational DB stores:

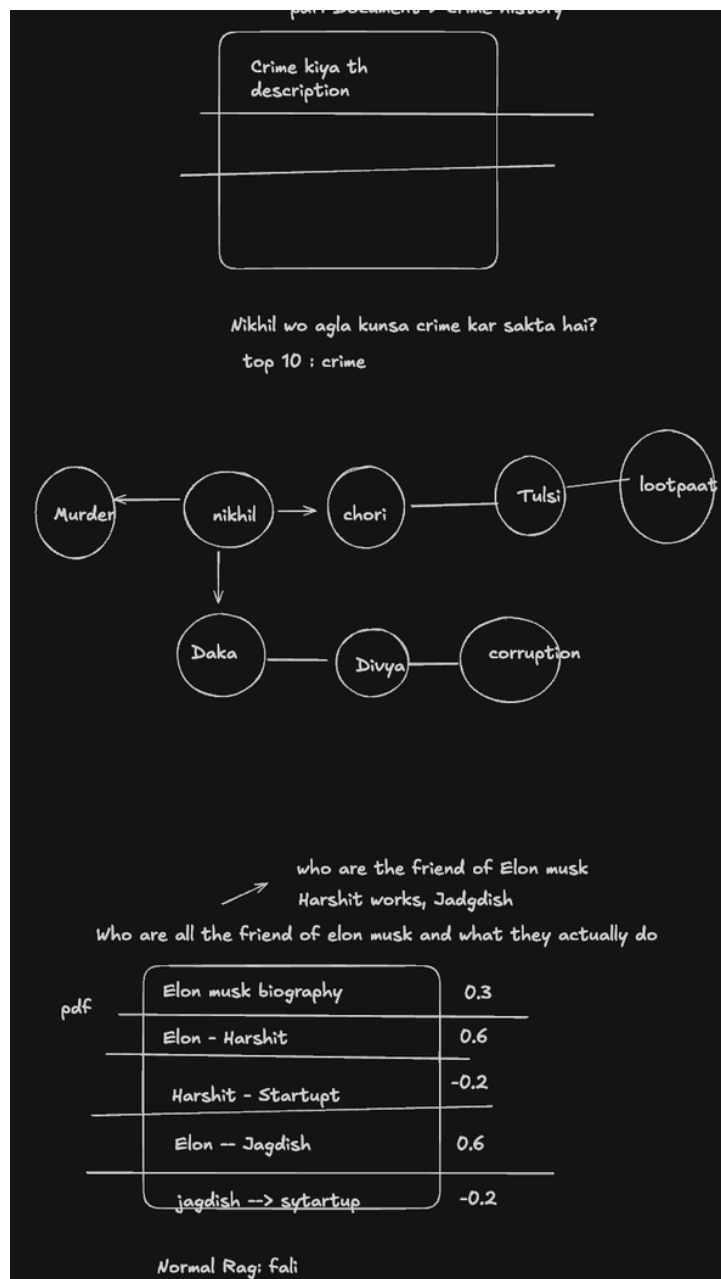
Data first → relationships derived using JOIN

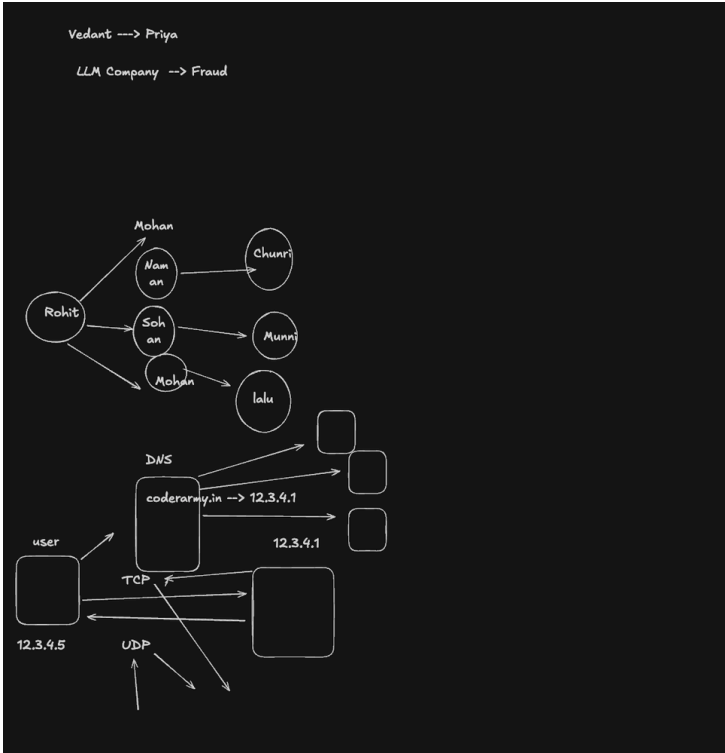
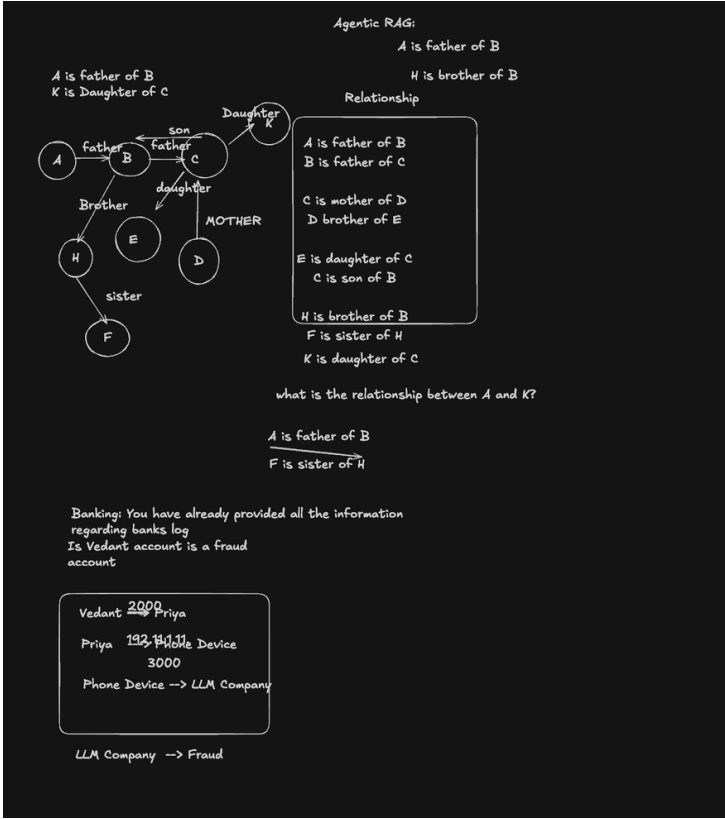
Graph DB stores:

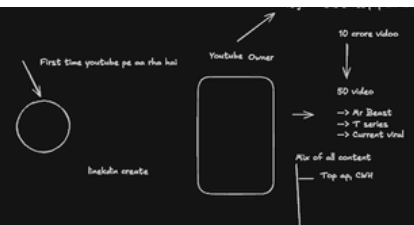
Relationships first-class citizens

That's why:

- It is extremely powerful for connected data.
- Traversal is $O(k)$ where k = number of connected nodes.
- Not dependent on total database size.

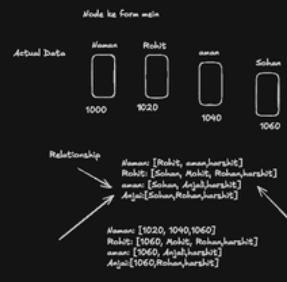
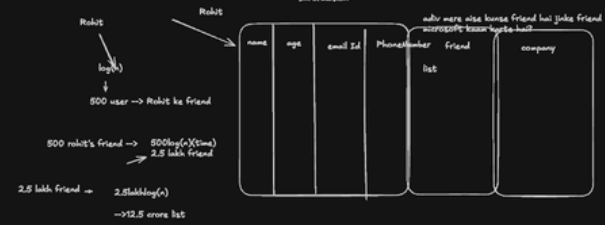






Rohit → 500 friend
 Vedant → 500 friend
 Adv → 500 friend
 Tishi → 500 friend

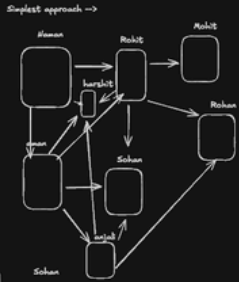
Rohit ke friend ke friend ke friend kya hai?
 List kaise do?
 Tishi mere aise kaise friend hai jo google ke andar kaha karte hai



Time complexity → $\log(x) + 500 * \log(x) + 2.5 \text{ lakh} * \log(x)$

Relationship: Interconnected hai

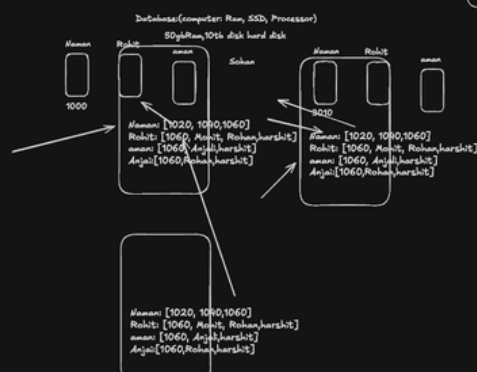
Simplest approach →



Index:
 Haman
 Rohit
 Sachin
 Aman
 Rohan
 Anjali

Relationship:
 Haman → Rohit
 Haman → Aman
 Rohit → Sachin
 Rohit → Rohan
 Sachin → Rohan
 Aman → Rohan
 Anjali → Rohan

GraphDB





computer desktop/rohit.mp4

computer desktop/rohit.mp4

Design

